Applicability of Non-Asymptotic Optimizations in Range Filters

Andrei Ogurtsov

April 9, 2026

Abstract

We implement and evaluate Approximate Range Emptiness (ARE) filters for LSM-tree storage systems. Starting from the information-theoretically optimal construction of Goswami et al. (SODA 2015), we develop a family of practical filters that explore different hash function designs: TODO

Contents

Chapter 1

Introduction

1.1 Notation

General. We denote by $\{0,1\}^L$ the set of all bit strings of length L.

Sets and parameters.

L	Key length in bits.
$U = \{0,1\}^L$	Universe of L-bit keys.
n	Number of stored keys.
$S \subseteq U$	Stored key set, $\lvert S\rvert = n$.
$Y = U \setminus S$	Non-keys (all points not in S).
$\mathcal{L}$	Maximum query range length.
ε	Target false positive rate.

Hash function and fingerprints.

h	Hash function $h\colon U \to U'$.
K	Fingerprint length in bits, $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$.
$U' = \{0,1\}^K$	Mapped universe.
$r = 2^K$	Mapped universe size.
$S' = h(S)$	Fingerprints of stored keys.
$Y' = h(Y)$	Images of non-keys under h.

Metrics.

FPR	False Positive Rate.
BPK	Bits Per Key: total filter size divided by n.

Structures.

ERE	Exact Range Emptiness — zero-error range filter.
ARE	Approximate Range Emptiness — range filter with one-sided error $\leq \varepsilon$.

1.2 Problem Statement

Let $U = \{0,1\}^L$ be a universe of L-bit keys. Given a set $S \subseteq U$ of n keys, a maximum query range length $\mathcal{L}$, and a target false positive rate ε, the *range emptiness problem* asks to build a data structure that answers "$[a,b] \cap S = \emptyset$?" for intervals of length $\leq \mathcal{L}$, with:

- **No false negatives:** if $[a,b] \cap S \neq \emptyset$, the answer is always "not empty".
- **Bounded false positives:** if $[a,b] \cap S = \emptyset$, the answer is "not empty" with probability at most ε.

Let $Y = U \setminus S$ denote all keys not stored in the structure.

1.2.1 Information-Theoretic Bounds

Goswami et al. [2015] establish the following bounds.

Theorem 1 (Lower bound, Goswami et al. [2015, §2])**.** *Any data structure solving the range emptiness problem requires at least* $n\big(\log_2(\mathcal{L}/\varepsilon) - c\big)$ *bits for an absolute constant* $c > 0$.

Remark 1. *The lower bound is worst-case over the input: it holds for every set* $S \subseteq U$ *with* $|S| = n$*, with no assumptions on the structure or distribution of* S*.*

Remark 2. *In practice, real datasets are often more regular. If the input is known to have additional structure, a data structure may use that structure to reduce the required space below the general bound.*

Theorem 2 (Upper bound, Goswami et al. [2015, §3])**.** *The lower bound of Theorem 1 is achieved up to an additive* $O(n)$ *term via two layers:*

1. *A locality-preserving hash (Section 4.1)* $h\colon U \to U'$*, where* $U' = \{0,1\}^K$ *and* $r = |U'| = 2^K$*. The hash partitions* U *into blocks of size* r*, applies an independent cyclic shift within each block, and then maps the shifted block onto* U' *by translation. Its pairwise collision probability is at most* $1/2^K$*.*
2. *An* ***Exact Range Emptiness (ERE)*** *(Section 2.1) structure over* $S' = h(S) \subseteq U'$*: a succinct structure with zero error and space* $n\log_2(r/n) + O(n)$ *bits, which answers "*$[a',b'] \cap S' = \emptyset$*?".*

The resulting space is $n\log_2(\mathcal{L}/\varepsilon) + O(n)$ bits, or equivalently $\log_2(\mathcal{L}/\varepsilon) + O(1)$ bits per key — matching the lower bound.

The fingerprint length $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$ controls accuracy: because each pairwise collision occurs with probability at most $1/2^K$, the overall error probability of the construction is bounded by $\varepsilon \leq n\mathcal{L}/2^K$. Thus, increasing K by one bit halves the false positive rate. After ERE compression (which subtracts $\log_2 n$ for block indexing), the effective cost per key is $K - \log_2 n = \log_2(\mathcal{L}/\varepsilon) + O(1)$

1.2.2 The Role of the Hash Function

The hash h projects U down to U'. Under this projection, the stored keys S map to fingerprints $S' = h(S)$, and the non-keys $Y = U \setminus S$ map to $Y' = h(Y)$.

A false positive occurs when Y' overlaps with S': a query point $y \in Y$ has $h(y) \in S'$, so the structure cannot distinguish it from a real key. We call the pre-images of a stored fingerprint $x' \in S'$ its **phantom points** — all keys y such that $h(y) = x'$. The fewer phantoms overlap with Y', the lower the FPR.

The ideal hash would map S' and Y' to disjoint regions of U' — zero overlap, zero false positives. In practice this is impossible within the space budget, but the choice of h determines how well S' and Y' separate for a given data distribution. This motivates two directions explored in this work: alternative hash function designs (Chapter 4) and data-adaptive segmentation strategies that bypass hashing entirely on dense regions (Chapter 5).

Chapter 2

Exact Range Emptiness

2.1 Succinct Representation (SODA 2015, §3.2)

The inner layer of every approximate filter is an *Exact Range Emptiness* (ERE) structure: given a universe $[r]$ of $\ell = \lceil \log_2 r \rceil$-bit keys and a set $S' \subseteq [r]$ of n keys, it answers "$[a, b] \cap S' = \emptyset$?" exactly (no false positives or false negatives). Throughout this chapter, S' denotes the input to ERE; in the ARE pipeline (Chapter 4), $S' = h(S)$.

2.1.1 Two-Level Hierarchy

To achieve the information-theoretic minimum of $n \log_2(r/n) + O(n)$ bits, the structure divides the universe $[r]$ into n equal-sized blocks:

Global level (succinct indexing). Two bit-vectors provide $O(1)$ navigation:

- D_1 (size n bits): $D_1[i] = 1$ if block i contains at least one key.
- D_2 (size $\sim 2n$ bits): an Elias–Fano [Elias, 1974, Fano, 1971] representation of block counts. Each non-empty block i contributes a 1 followed by n_i zeros. A $\textsc{Select}_1$ / $\textsc{Rank}_0$ pair maps any block index to its starting position in the data array in $O(1)$ time.

Local level (bit-packed suffixes). Instead of storing full keys, only the suffix $x \bmod (r/n)$ is stored for each key x. The suffix length is $W = |r| - \lceil \log_2 n \rceil$ bits. All suffixes are packed into a dense `uint64` array using bit-shifts, eliminating per-element overhead.

2.1.2 Local Search in Buckets

The original paper [Goswami et al., 2015] suggests a Weak Prefix Search structure (Hollow Z-Fast Trie) for $O(1)$ worst-case local queries. We use binary search on bit-packed suffixes instead, with an adaptive fallback to linear scan for small buckets. Section 2.1.3 details why.

Block occupancy under uniform distribution. The structure divides the universe into $m = 2^{\lfloor \log_2 n \rfloor}$ blocks; each non-empty block stores its keys' suffixes in a packed array (a *bucket*). The expected number of keys per block is $\lambda = n/m \in [1, 2)$. When n is a power of two, $m = n$ and $\lambda = 1$ exactly. Each key falls into a uniformly random block, so the occupancy of a fixed block follows $\text{Bin}(n, 1/m)$, which converges to $\text{Poisson}(\lambda)$ as $n \to \infty$ [Mitzenmacher and Upfal, 2017, Ch. 5].

For $\lambda = 1$, the fraction of empty blocks converges to $e^{-1} \approx 36.8\%$, and the mean occupancy among non-empty blocks to $1/(1 - e^{-1}) \approx 1.58$.

To estimate the maximum bucket size, we use a union bound. The probability that a fixed block contains at least k keys is $\Pr[\text{Poisson}(\lambda) \geq k] = 1 - \sum_{j=0}^{k-1} e^{-\lambda}\lambda^j/j!$ (which simplifies to $1 - \sum_{j=0}^{k-1} e^{-1}/j!$ when $\lambda = 1$). The probability that *any* of the m blocks contains $\geq k$ keys is at most

$$\Pr\big[\max_i X_i \geq k\big] \;\leq\; m \cdot \Pr[\text{Poisson}(\lambda) \geq k].$$

The expected number of blocks with occupancy $\geq k$ drops below 1 when $m \cdot \Pr[\text{Poisson}(\lambda) \geq k] < 1$. In the tables below we use $\lambda = 1$ ($m = n$, powers of two). Table 2.1 shows that this threshold closely matches the measured maximum.

n	Empty (%)	Avg / non-empty	Max (meas.)	k^*	$n \cdot \Pr[X \geq k^*]$
2^{20}	36.75	1.58	9	10	0.12
2^{24}	36.79	1.58	9	11	0.17
2^{27}	36.79	1.58	10	12	0.11
2^{30}	36.79	1.58	13	12	0.89

Table 2.1: ERE bucket statistics for uniform keys with $\lambda = 1$ ($m = n$). k^* is the smallest k such that $n \cdot \Pr[\text{Poisson}(1) \geq k] < 1$. The measured maximum can exceed k^* (e.g. $n = 2^{30}$: max $= 13 > k^* = 12$) because the bound is not tight; $n \cdot \Pr[\text{Poisson}(1) \geq 13] \approx 0.07$ at $n = 2^{30}$ (see Table 2.2).

Table 2.2 shows how rapidly the tail probability drops with k: even at $n = 2^{30}$, the probability of any block containing ≥ 20 keys is below 10^{-10}, making buckets larger than $\sim$15 keys virtually impossible for uniform data.

k	$\Pr[\text{Poisson}(1) \geq k]$	$n \cdot \Pr[X \geq k]$, $n = 2^{30}$
10	1.1×10^{-7}	1.2×10^{2}
13	6.4×10^{-11}	6.8×10^{-2}
15	3.0×10^{-13}	3.2×10^{-4}
20	1.6×10^{-19}	1.7×10^{-10}
25	2.5×10^{-26}	2.6×10^{-17}

Table 2.2: Poisson tail probabilities for bucket occupancy ($\lambda = 1$), computed via $\Pr[X \geq k] = 1 - \sum_{j=0}^{k-1} e^{-1}/j!$. At $k = 13$ the expected number of such blocks across all $n = 2^{30}$ blocks is $\sim$0.07 — consistent with observing max $= 13$ occasionally. At $k = 20$ it is below 10^{-10} even for $n = 2^{30}$.

Non-uniform distributions and worst-case bounds. For non-uniform input, the Poisson model does not apply: clustered keys concentrate in a few blocks. When ERE is used inside SODA ARE (Section 4.1), the locality-preserving hash $h(x) = (u + x) \bmod 2^K$ (where u is a per-block constant) maps consecutive keys to consecutive hashed values, preserving local density.

However, the bucket size is always bounded by the block capacity 2^w, where $w = K - \lfloor \log_2 n \rfloor$ is the suffix length and K is the fingerprint width in bits (defined in Section 4.1). For industry-standard space budgets (BPK ≤ 15; since BPK $\approx w + 3.2$ from Table 2.5, this gives $w \leq 12$), so the maximum bucket contains at most $2^{12} = 4096$ keys. At $w = 12$ bits, such a worst-case bucket occupies $4096 \times 12/8 = 6144$ bytes — well within L1 cache (typically 32–64 KB).

Search strategy. We implement both binary search ($O(\log k)$) and linear scan ($O(k)$) over bit-packed suffixes in a bucket of k keys. Benchmarks on Apple M4 Max show that linear scan is $\sim$3$\times$ faster for $k \leq 12$ (1.3–2.7 ns vs. 5–9 ns) due to sequential prefetch, while binary search wins for $k > 128$ (21 ns vs. 35 ns at $k = 256$). The crossover is at $k \approx 128$ and is stable across suffix widths $w \in \{8, 11, 16, 20\}$.

The default query uses adaptive dispatch: linear scan for buckets with $k \leq 128$, binary search above. The total ERE query takes $\sim$36 ns on Apple M4 Max (stable across $n \in [2^{10}, 2^{20}]$ and key widths 64–512 bits), of which $\sim$27–35 ns is Rank/Select overhead on D_1 and D_2. Bucket search contributes at most $\sim$9 ns (binary, $k = 12$) or $\sim$2.7 ns (linear, $k = 12$) — roughly 7–25% of the total. This makes the choice between search strategies a micro-optimization; the key property is that even in the worst case ($k = 2^w$, binary search), query time grows only logarithmically.

2.1.3 Why Not Weak Prefix Search

Goswami et al. [2015] propose $O(1)$ worst-case local queries via a Weak Prefix Search structure built from a Hollow Z-Fast Trie (HZFT) and Monotone Minimal Perfect Hashing (MMPH). The authors analyse these components in a word-RAM model where hash table lookups and trie navigation cost $O(1)$, and the space is expressed up to $O(n)$ additive terms. This is sufficient for a theoretical result, but the hidden constants become dominant in practice.

We implemented three variants of Weak Prefix Search, collectively referred to as LERLOC (**L**ocal **E**xact **R**ange **LOC**ator), differing in the trie and MMPH components used:

Variant	Space (bits/key)	Query (ns), L=64
LERLOC Fast (HZFT + BoomPHF)	179	1095–1466
LERLOC Compact (SHZFT + BoomPHF)	151	372–567
LeMon-LERLOC (SHZFT + LeMonHash)	59	537–762

Table 2.3: LERLOC variants: space and query latency on Apple M4 Max, $n = 1024$–32768, 64-bit keys. LeMon-LERLOC breakdown: SHZFT trie $\sim$35 BPK, LeMonHash MMPH $\sim$19 BPK, RSDic leaf bitvector $\sim$5 BPK. The MMPH indexes the boundary set P of the trie: 3 strings per node (n leaves $+(n-1)$ internal), so $|P| \leq 3(2n-1) = 6n-3$ before deduplication; empirically $|P|/n \approx 4.3$ for $n > 10^5$.

The high space cost stems from three expansion factors hidden behind $O(n)$ in the paper: (1) the SHZFT hash table stores not only $2n-1$ true descriptors but also $\sim$3.5n pseudo-descriptors required by Fat Binary Search, totalling $\sim$5.5n entries at $\sim$35 BPK; (2) the range locator boundary set P contains 3 strings per trie node ($\leq 6n-3$ before deduplication, empirically $\sim$4.3n); (3) the MMPH on P stores a `uint8` rank per element (8 bits $\times\, 4.3 \approx 34$ BPK). These factors stack: the total "$O(n)$ additional space" becomes $\sim$100 bits/key in practice (see the detailed breakdown in `locators/rloc/MEMORY_INVESTIGATION.md`).

When ERE is used inside an ARE filter, the suffix width is $w = K - \lfloor \log_2 n \rfloor \approx \lceil \log_2(\mathcal{L}/\varepsilon) \rceil$ (since $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$), which depends only on the range length $\mathcal{L}$ and FPR ε, not on n. For typical parameters ($\mathcal{L} \leq 64$, $\varepsilon \geq 0.01$), w is 7–12 bits. The data itself costs w bits per key; even the most compact LERLOC variant (59 BPK) adds 5–8$\times$ more space than the data it indexes.

Query latency is equally problematic. LERLOC replaces only the local search inside a bucket, while Rank/Select navigation on D_1 and D_2 ($\sim$27 ns) remains unchanged. Each LERLOC call descends the trie (pointer chasing through hash tables), then performs an MMPH

rank lookup — at least 370 ns for the Compact variant at $L = 64$. Binary search on a typical bucket of ∼12 keys takes 5–9 ns (Section 2.1.2), making LERLOC 40–110× slower for the operation it replaces.

The root cause is that the $O(1)$ theoretical guarantee hides large constants: each "constant-time" trie step involves hashing a variable-length prefix, probing a hash table, and following a pointer — operations that are individually cheap but accumulate across $O(\log \log r)$ trie levels. For the small buckets that arise in practice (≤13 keys for uniform data, $\leq 2^w$ in general), binary search on packed suffixes is both faster and requires no additional space.

Even in an adversarial worst case — every key in a single bucket — binary search remains fast:

w (suffix bits)	Bucket size (2^w)	Binary search (ns)
12	4 096	37
15	32 768	48
17	131 072	58

Table 2.4: Worst-case binary search on a fully packed bucket (Apple M4 Max). At $w = 12$ (BPK ≈ 15), the entire bucket search costs 37 ns — still 10× faster than the best LERLOC variant.

2.1.4 Space Analysis

ℓ (key bits)	Suffix bits ($\lvert r\rvert - \log_2 n$)	Metadata	Total BPK
64	44.07	3.2	47.27
128	108.07	3.2	111.27
256	236.07	3.2	239.27
512	492.07	3.2	495.27

Table 2.5: Measured ERE space with $n = 10^6$ keys. Metadata ($D_1 + D_2$ + rank/select indices) is constant at ∼3.2 BPK.

Space grows linearly with ℓ — unavoidable for an exact structure, since it must store the information distinguishing the keys. This linear dependence motivates the move to approximate range emptiness: by storing hashed fingerprints of length $K \approx \log_2(\mathcal{L}/\varepsilon)$ instead of full keys, the space becomes independent of the original key length ℓ.

2.1.5 Complexity Summary

- **Build:** $O(n \cdot |r|)$ — a single pass over sorted keys.
- **Query:** $O(|r|/w)$ expected (block index + binary search over ∼2 keys of ℓ bits each); $O((|r|/w) \log n)$ worst case if all keys collide in a single block. When used inside ARE with K-bit fingerprints fitting in one machine word, this reduces to $O(1)$ expected.
- **Space:** $n \log_2(r/n) + O(n)$ bits, matching the information-theoretic lower bound.

Chapter 3

Key Distributions

3.1 Key Distributions in Practice

The theoretical bounds of Section 1.2.1 hold for any key set. In practice, however, keys in storage systems are not arbitrary — they follow distributions shaped by the application domain. Understanding these distributions is essential for evaluating hash function designs and segmentation strategies in the following chapters.

3.1.1 The Core Assumption

We assume that queries target regions of the key space where stored keys are dense, rather than arbitrary empty regions.

Remark 3. *This holds for most workloads, where queries reflect the application state and therefore concentrate on regions that contain data.*

This assumption has two consequences:

1. FPR in sparse regions of the key space matters less — queries rarely land there.
2. Dense regions dominate the effective FPR experienced by the application.

The SODA hash (Section 4.1) ignores this assumption entirely — it provides worst-case guarantees for any distribution. The truncation hash (Section 4.2) performs well only under uniformity. The hybrid approaches (Chapter 5) explicitly exploit it by isolating dense regions.

3.1.2 Common Industrial Distributions

Keys in LSM-tree storage systems typically exhibit one or more of the following patterns:

Sequential / near-sequential. Auto-incrementing primary keys, monotonic timestamps, log sequence numbers. Keys are approximately evenly spaced with small gaps. The key space is dense — spread is much smaller than the universe.

Clustered. Keys concentrate in a few dense regions separated by large gaps. Examples:

- **User activity:** a social network stores events keyed by user ID prefix + timestamp. Active users produce dense clusters; inactive users leave gaps.

- **Geospatial data:** S2 cell IDs [Google, 2017] — a hierarchical spatial index that maps geographic coordinates to 64-bit integer keys — cluster around populated areas. Rural regions are sparse.
- **Time-partitioned data:** recent partitions are dense (active writes), older partitions are sparse (archived).

High-entropy / uniform-like. UUIDs, cryptographic hashes, randomly generated tokens. Keys are spread across the full key space with no exploitable structure. This is the hardest case for distribution-aware optimizations and the only case where truncation achieves its theoretical BPK.

3.1.3 SOSD Benchmark Datasets

For empirical evaluation, we use real-world datasets from the SOSD (Search on Sorted Data) benchmark [Marcus et al., 2020], masked to 60-bit keys:

Dataset	n (full)	Character
Facebook	200M	User IDs (uint64). Highly clustered — dense regions of sequential IDs with large gaps between ID ranges.
Books	200M	ISBN sequences (uint32). Clustered by publisher prefix, near-sequential within each publisher.
Wiki	200M	Wikipedia edit timestamps (uint64). Moderately clustered — bursts of activity during events, quieter periods between.
OSM	800M	OpenStreetMap cell IDs (uint64). Geospatially clustered — dense in populated areas, sparse in oceans and deserts.

Table 3.1: SOSD datasets used in our evaluation. All exhibit clustering to varying degrees — none are uniform. Benchmarks in Chapter 6 use subsets of these datasets (e.g. $n = 2^{24}$), masked to 60-bit keys.

3.1.4 Synthetic Distributions

To isolate the effect of specific distribution properties, we also generate synthetic key sets:

- **Uniform:** keys drawn uniformly from $[0, 2^{60})$. Average gap $\approx 2^{60}/n$. No clustering. Baseline for truncation-friendly data.
- **Clustered:** a mixture of Gaussian clusters with controlled parameters (number of clusters, intra-cluster spread, inter-cluster gap). Models the typical industrial pattern.

Together, the SOSD and synthetic datasets span the spectrum from highly clustered (Facebook, Books) through moderately structured (Wiki, OSM) to structureless (uniform random) — allowing us to evaluate each filter design across the range of distributions it may encounter in practice.

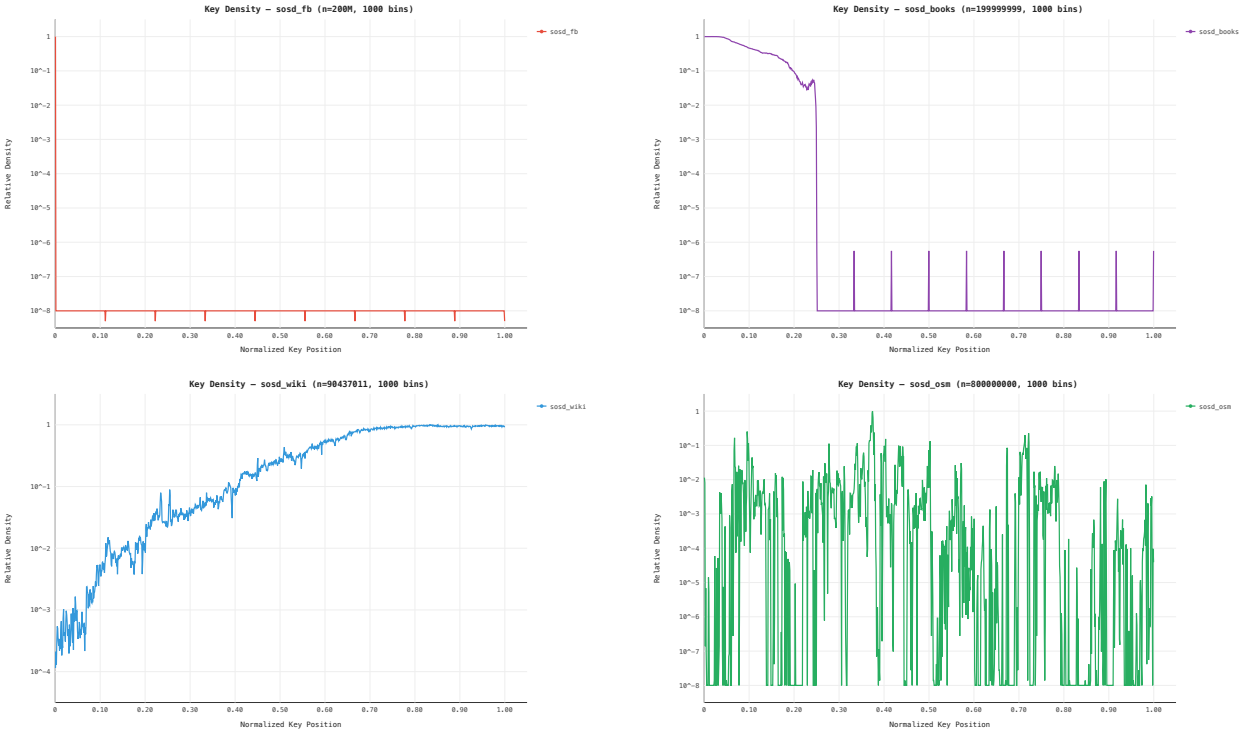

Figure 3.1: Key density histograms for SOSD datasets (1000 bins, log-scale Y-axis). Facebook and Books concentrate keys in a narrow region of the key space. Wiki shows a smooth density gradient. OSM is geospatially clustered with many empty regions.

Chapter 4

Hash Function Approaches

4.1 Pairwise-Independent Hash (SODA Baseline)

4.1.1 The Hash Function

The paper's original locality-preserving hash [Goswami et al., 2015, §3.1]:

$$h(x) = \big(u(\lfloor x/r \rfloor) + x\big) \bmod r, \quad r = 2^K \tag{4.1}$$

where $u : [U/r] \to [r]$ is drawn from a pairwise-independent family, stored as two 64-bit coefficients (a, b).

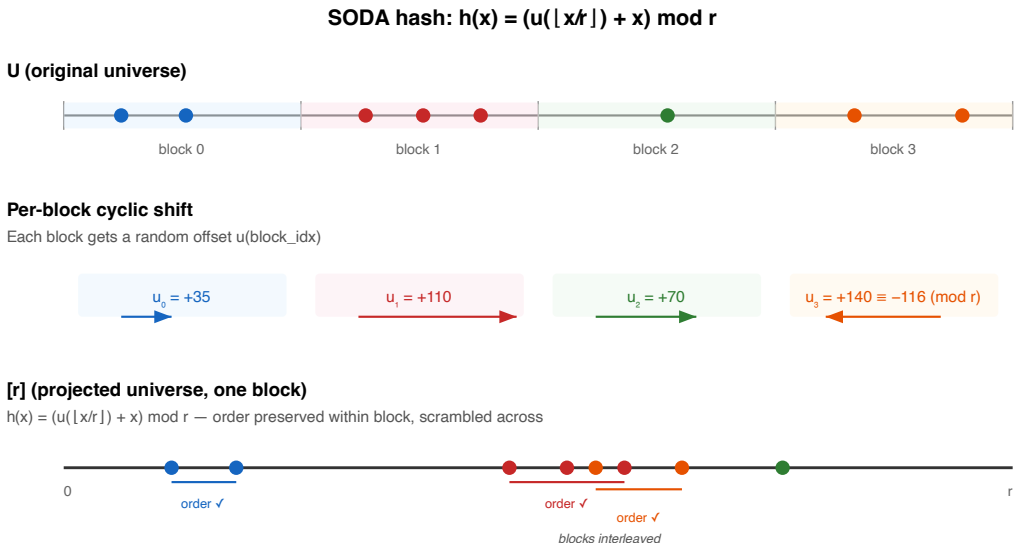

Figure 4.1: SODA hash mechanism: per-block cyclic shift maps keys to $[0, 2^K)$. A query range spanning two blocks produces at most two intervals in the mapped space.

Properties of the hash:

- **Pairwise-independent:** $\Pr[h(x_1) = h(x_2)] \leq 1/r$ regardless of key structure. This holds for sequential, clustered, or adversarial data.
- **Locality-preserving:** the hash applies a per-block cyclic shift, so consecutive keys within the same block map to consecutive positions in $[r]$. A query range $[a, b]$ spans at most 2 blocks, producing at most 2 intervals in the mapped space.
- **Compact:** stores only two 64-bit coefficients (a, b) — $O(1)$ bits, independent of n.

4.1.2 ARE Construction With SODA Hash

This is a direct implementation of the construction from Goswami et al. [2015, §3]. The hash design, its proofs of locality-preservation and pairwise independence, and the resulting space/FPR bounds are all from the original paper — our contribution is the practical implementation.

Combining this hash with ERE (Chapter 2) yields the baseline ARE filter:

1. Divide U into blocks of size $r = 2^K$. Let $\text{blockIdx}(x) = \lfloor x/r \rfloor$ be the block index of key x.
2. For each block, compute pairwise-independent shift: top K bits of $(a \cdot \text{blockIdx}(x) + b)$.
3. Hash each key via Equation (4.1).
4. Sort, deduplicate, build ERE over $[0, 2^K)$.
5. Query: hash both endpoints; cyclic shift may split the range into two intervals — check both in ERE.

Filter properties.

- **FPR $\leq \varepsilon$ for any data distribution** — follows from the pairwise independence of the hash.
- **No false negatives** — follows from the locality-preserving property: all images of the query range are checked in ERE.
- **Space:** $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$, giving BPK $= \log_2(\mathcal{L}/\varepsilon) + O(1)$ — matching the information-theoretic lower bound. Including ERE metadata (D_1, D_2), the total cost is $\log_2(\mathcal{L}/\varepsilon) + \sim 3$ bits per key.

4.1.3 Empirical Validation

On uniform data ($n = 2^{18}$, $\mathcal{L} = 128$), the measured FPR-vs-BPK curve tracks the theoretical bound with a gap of $\sim$3 BPK (ERE overhead from D_1, D_2, Figure 4.2). On non-uniform data with closely spaced keys the gap narrows to $\sim$1 BPK: hash collisions among nearby keys reduce the number of unique fingerprints, making ERE more compact per key (Figure 4.3).

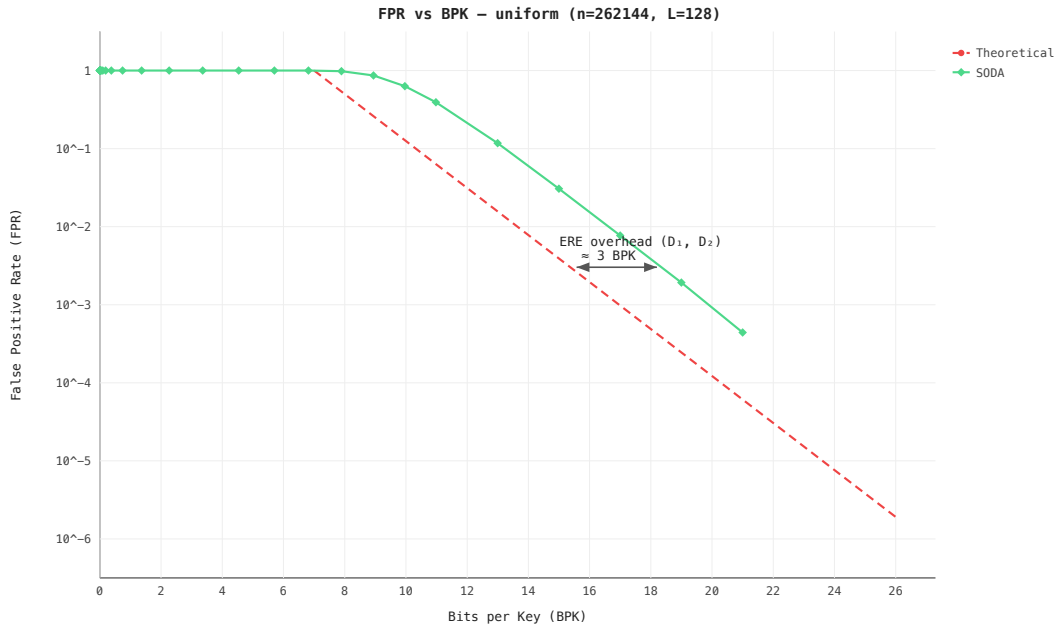

Figure 4.2: FPR vs BPK on uniform keys ($n = 2^{18}$, $\mathcal{L} = 128$). SODA tracks the theoretical bound with ∼3 BPK overhead from ERE metadata.

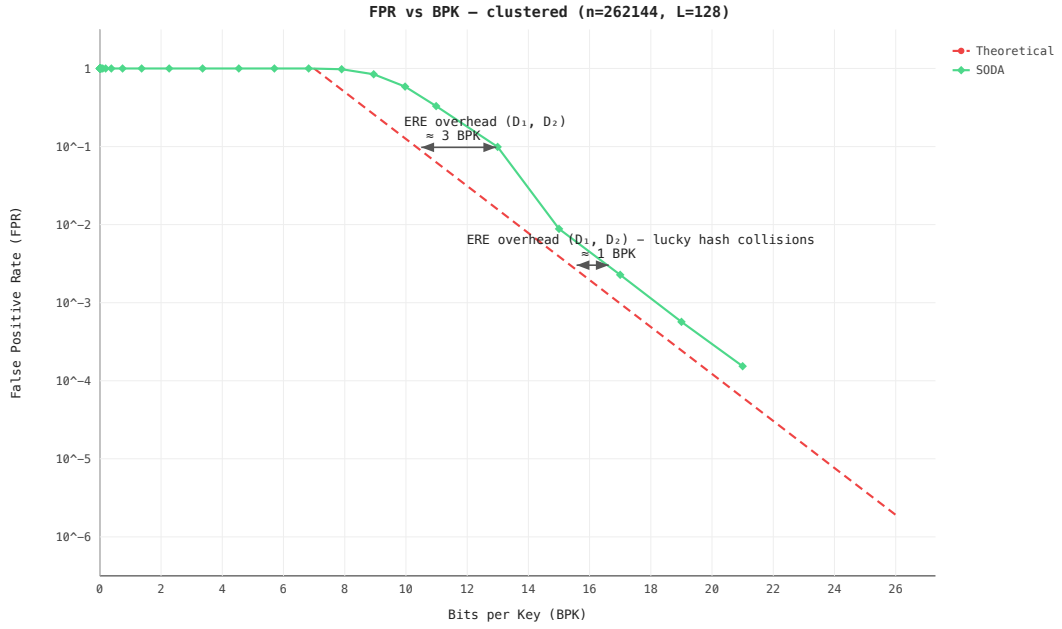

Figure 4.3: FPR vs BPK on clustered keys ($n = 2^{18}$, $\mathcal{L} = 128$). The gap narrows to ∼1 BPK as hash collisions among nearby keys reduce the effective number of unique fingerprints.

4.2 Prefix Truncation

4.2.1 The Hash Function

An alternative locality-preserving hash: $h(x) = \lfloor x/2^t \rfloor$, keeping the top $K = L - t$ bits of each key.

Properties of the hash:

- **Locality-preserving:** truncation preserves order — if $a < b$, then $h(a) \leq h(b)$. Intervals map to intervals.
- **Contiguous phantoms:** the 2^t phantom points of each stored key x form an interval of length 2^t in the original universe — all points sharing the same K-bit prefix. This contrasts with the SODA hash, where phantoms are scattered uniformly.
- $O(1)$ **storage:** the hash is a bit-shift — stores only the shift amount t.

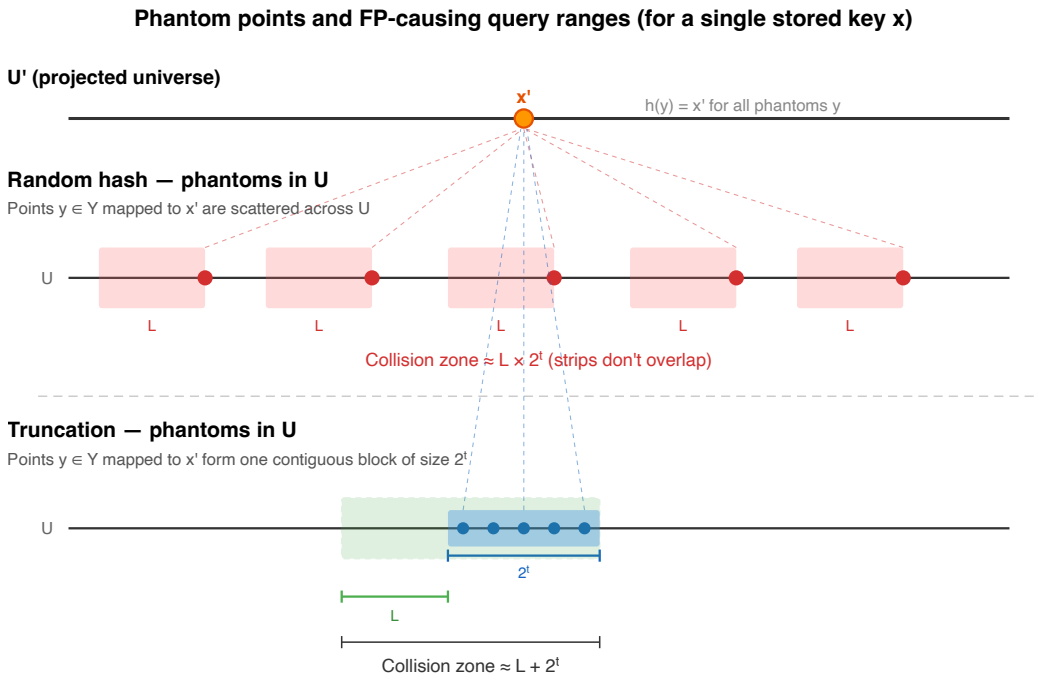

Figure 4.4: Phantom point distribution: random hash (SODA) scatters phantoms uniformly across U; truncation concentrates them in a contiguous block adjacent to the key.

4.2.2 ARE Construction With Truncation

Combining truncation with ERE yields a filter where the contiguous phantom structure (Figure 4.4) changes the collision analysis:

- **SODA hash:** phantoms scattered $\Rightarrow$ collision zone per key is $\mathcal{L} \times 2^t$.
- **Truncation:** phantoms contiguous $\Rightarrow$ collision zone per key is $\mathcal{L} + 2^t$.

The additive (rather than multiplicative) interaction eliminates the $\mathcal{L}$ factor from the filter's space formula:

Hash	Collision zone	K for FPR $\leq \varepsilon$	Filter BPK
Random (SODA)	$\mathcal{L} \times 2^t$	$\log_2(n\mathcal{L}/\varepsilon)$	$\log_2(\mathcal{L}/\varepsilon) + O(1)$
Truncation	$\mathcal{L} + 2^t$	$\log_2(2n/\varepsilon)$	$\log_2(1/\varepsilon) + O(1)$

Table 4.1: Choosing truncation over the SODA hash reduces the resulting filter's BPK by $\log_2(\mathcal{L})$. At $\mathcal{L} = 128$, this is a saving of 7 bits per key.

4.2.3 The Price of Contiguity

The same property that reduces the filter's space is also the fundamental weakness: all false positives are concentrated near stored keys. For a stored key x, its phantom block — the 2^t consecutive points sharing the same K-bit prefix as x — is the only region that can produce false positives.

When inter-key gaps are smaller than the phantom size 2^t, phantom intervals of neighboring keys overlap and FPR approaches 100%. This occurs with:

- **Sequential keys** $(x, x{+}1, x{+}2, \ldots)$: every gap is covered by phantoms from both sides.
- **Clustered keys**: phantom intervals overlap, covering the cluster without gaps.

Only uniformly spread keys with gaps $\gg 2^t$ achieve the theoretical FPR — a narrow use case (Figure 4.5). The SODA hash avoids this entirely: pairwise independence scatters phantoms uniformly, giving $\Pr[h(x_1) = h(x_2)] \leq 1/r$ for any distribution, at the cost of $\log_2(\mathcal{L})$ extra bits per key in the resulting filter.

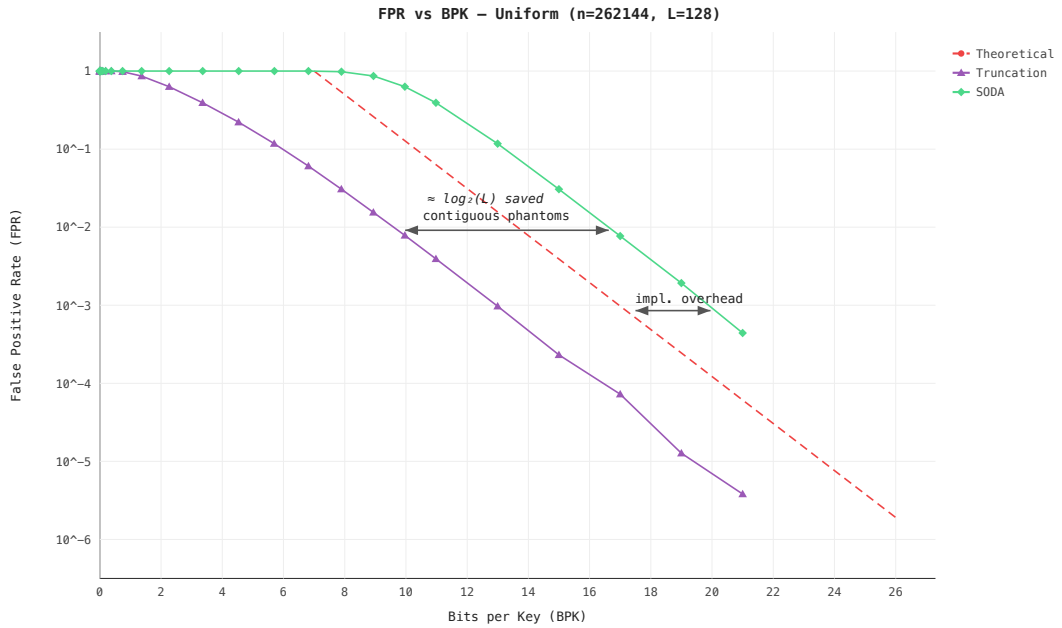

Figure 4.5: FPR vs BPK for truncation vs SODA on uniform keys ($n = 2^{18}$, $\mathcal{L} = 128$). The SODA curve is shifted right by $\approx \log_2(128) = 7$ BPK — matching the theoretical prediction.

4.3 Adaptive Filter

The adaptive filter checks at build time whether hashing is necessary at all.

4.3.1 Exact Mode

1. **Normalize:** subtract $\min(S)$ from all keys, shifting the dataset to start at 0.
2. **Measure spread:** $M = \lceil \log_2(\max(S) - \min(S)) \rceil$.
3. **Decide:**
   - $M \leq K$: the entire dataset fits in K bits $\Rightarrow$ **exact mode**. Build ERE directly over normalized keys. No hash, FPR = 0.
   - $M > K$: data too spread $\Rightarrow$ **SODA mode**. Apply pairwise-independent hash, same filter guarantees as Section 4.1.

4.3.2 When Does Exact Mode Trigger?

Exact mode requires $\max(S) - \min(S) < 2^K = n\mathcal{L}/\varepsilon$. Rewriting in terms of density $\rho = (n-1)/(\max(S) - \min(S))$:

$$\rho > \frac{\varepsilon}{\mathcal{L}} \tag{4.2}$$

or equivalently, the average gap $\bar{g} = (\max(S) - \min(S))/(n-1)$ must satisfy $\bar{g} < \mathcal{L}/\varepsilon$.

$\mathcal{L}$	ε	Max avg gap	Min density
128	10^{-3}	128,000	7.8×10^{-6}
128	10^{-6}	1.28×10^{8}	7.8×10^{-9}
16	10^{-3}	16,000	6.25×10^{-5}

Table 4.2: Exact mode threshold for typical parameters. The density requirement is extremely low — exact mode fails only when keys are scattered across a universe much wider than $n \cdot \mathcal{L}/\varepsilon$.

This becomes especially powerful in combination with hybrid segmentation (Chapter 5): each dense cluster is isolated into its own adaptive filter, where the local spread is small enough for exact mode, giving FPR = 0 on the dense parts of the data.

4.4 CDF Mapping (Experimental)

An experimental approach: use the empirical CDF of the data as the locality-preserving hash h, redistributing the FPR budget from sparse regions to dense regions.

Status: theoretical exploration. Benchmarks did not show practical improvement over the SODA hash or hybrid segmentation.

4.4.1 Motivation

The information-theoretic lower bound (Theorem 1) holds for all data and query distributions. But if queries follow approximately the same distribution as stored keys — users query ranges where data exists, not empty regions — then FPR in sparse inter-cluster gaps does not matter. If we could *move* FPR budget away from dense regions and into sparse regions, the effective FPR experienced by real queries would be lower for the same number of bits.

4.4.2 The Hash Function

The CDF is a monotonic function that maps keys to a near-uniform distribution (Figure 4.6):

- **Where keys are dense** (clusters): the mapped space is stretched, so the same number of non-keys is spread over a wider interval ⇒ fewer non-keys per unit length ⇒ lower local FPR.
- **Where keys are sparse** (gaps): the mapped space is compressed, non-keys pile up ⇒ higher local FPR. But under our assumption, queries rarely land here.

Monotonicity ($x_1 < x_2 \Rightarrow h(x_1) \leq h(x_2)$) preserves range queries — intervals map to intervals.

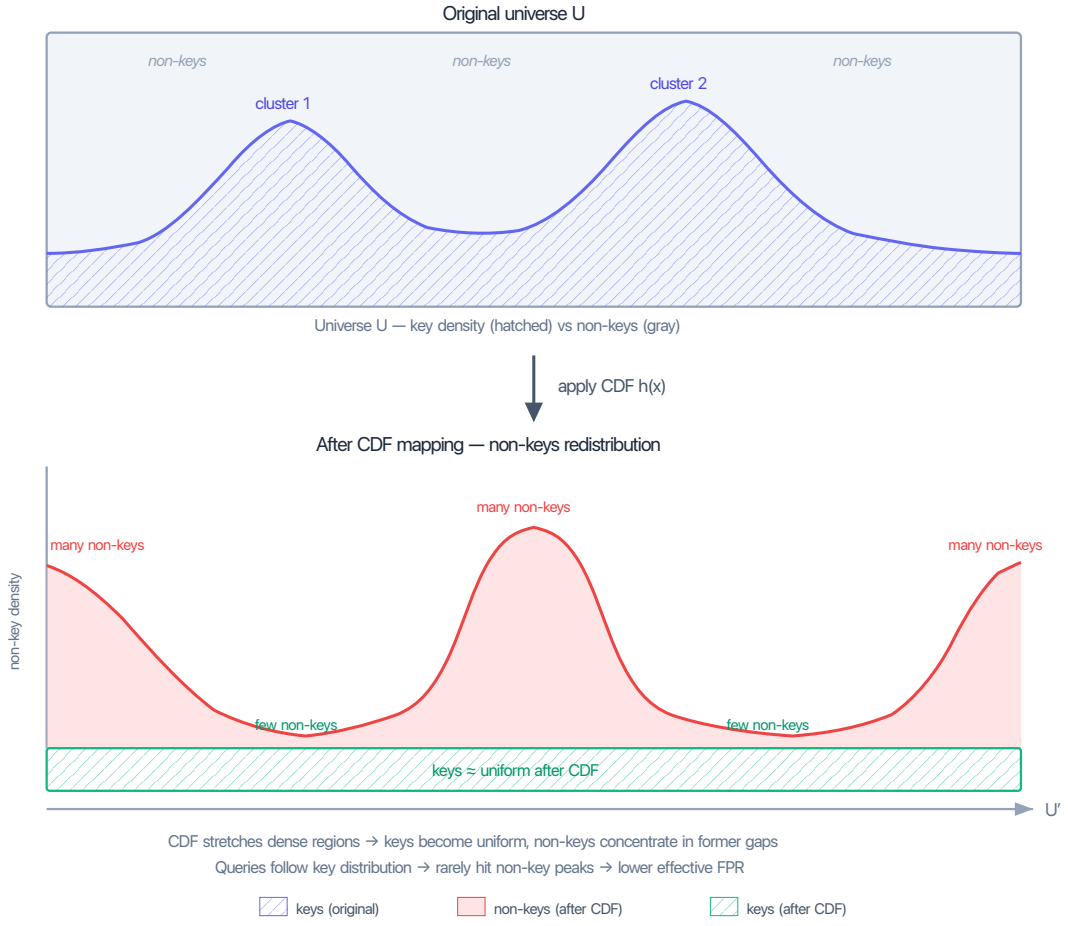

Figure 4.6: CDF redistribution. Top: original universe with non-uniform key density (uniform background + two clusters). After CDF mapping: keys become near-uniform, non-key density inverts — valleys where clusters were (low FPR), peaks where gaps were (high FPR, but queries rarely land there).

Important caveat: because the CDF maps everything monotonically, keys and non-keys are still interleaved in U' — there is no strict separation. The FPR reduction is not formally bounded and depends entirely on the query distribution matching the data distribution.

4.4.3 ARE Construction With CDF Hash

1. Sort input keys.

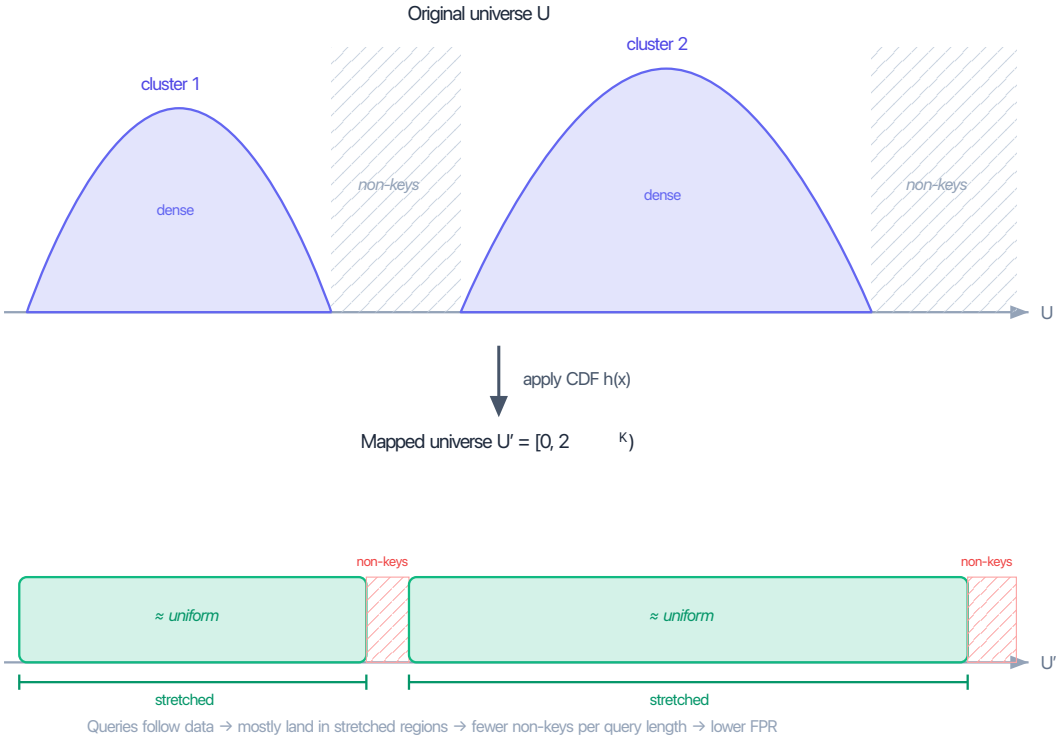

Figure 4.7: Before and after CDF mapping. Dense clusters (top) are stretched into uniform blocks (bottom); sparse gaps are compressed.

2. Build a PGM index [Ferragina and Vinciguerra, 2020] on float64-converted keys to obtain an approximate CDF.
3. Fix monotonicity (PGM positions may be non-monotone due to float64 rounding) via running max.
4. Sample CDF control points every `pgmEpsilon` keys ⇒ piecewise-linear model.
5. Compute L_{eff}: the effective range length after CDF distortion. A query of length $\mathcal{L}$ maps to a range of variable width depending on the local CDF slope, bounded by $L_{\text{eff}} = \mathcal{L} \times \max_{\text{segment}} \text{density}$.
6. Set $K = \lceil \log_2(n \cdot L_{\text{eff}}/\varepsilon) \rceil$.
7. Map keys through the piecewise-linear CDF, deduplicate, build ERE.
8. Query: map both endpoints through the same CDF, forward to ERE.

CDF model cost: each control point stores a (key, rank) pair = 128 bits. With `pgmEpsilon` = 128: $\approx n/128$ points ≈ 1 bit per key.

4.4.4 Limitations

- $O(n^2)$ build time due to PGM convex hull construction; constructor returns error for $n > 2^{20}$.
- Float64 precision loss for keys $> 2^{53}$.
- FPR is not formally bounded — depends on query/data distribution match.
- CDF model overhead adds 1–2 BPK on top of ERE storage.

4.4.5 Experimental Results

Benchmarks against the SODA hash showed that CDF-ARE does not provide a consistent practical advantage:

- When query distribution matches data: CDF-ARE uses fewer bits per key, but FPR is comparable or worse due to L_{eff} amplification in dense regions.
- When query distribution diverges from data: FPR degrades catastrophically (40–58% at $\sigma_{\text{query}} = 4 \times \sigma_{\text{data}}$).
- The $O(n^2)$ build cost makes it impractical for large datasets.

Hybrid segmentation (Chapter 5) achieves a similar goal — exploiting clustering — more effectively via segmentation and exact mode, without the CDF overhead.

Chapter 5

Hybrid Segmentation

A single global filter must handle the entire key space — if the spread exceeds 2^K, hashing is unavoidable and FPR is nonzero everywhere. But real-world keys are not spread uniformly: they concentrate in dense clusters separated by large gaps (Section 3.1).

The hybrid idea is to *segment* the key set: isolate each dense cluster into its own sub-filter, where the local spread is small enough for exact mode (FPR = 0, Section 4.3), and route the remaining sparse keys to a fallback filter that uses hashing. Dense regions — where most queries land — pay zero false positives; only the sparse tail incurs FPR $\leq \varepsilon$.

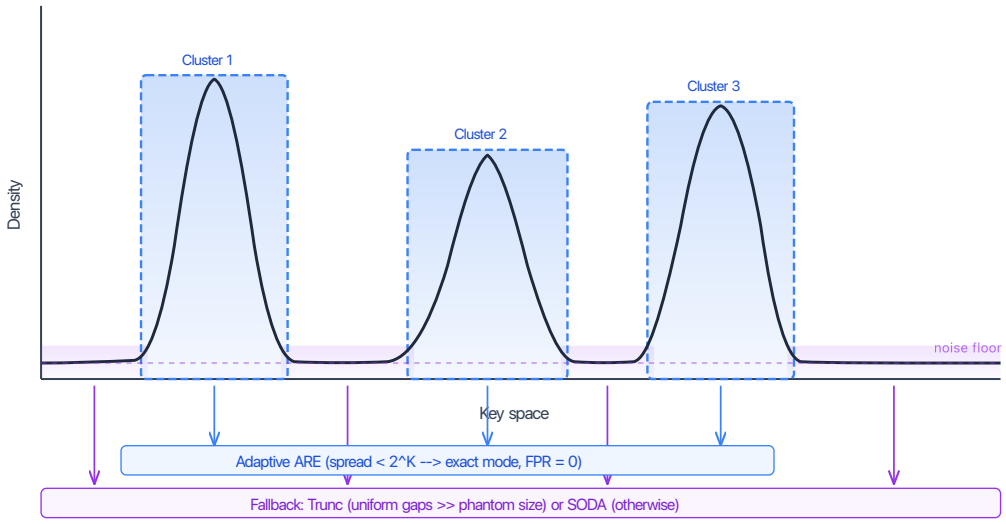

Figure 5.1: Hybrid segmentation overview: dense clusters are isolated into exact-mode sub-filters (FPR = 0), while sparse keys go to a fallback filter (truncation or SODA). The segmentation method differs between approaches — gap-percentile (Section 5.1) vs 1D DBSCAN (Section 5.2) — but the architecture is the same.

Why Clusters Compress So Well

Real-world key distributions concentrate most keys into a few tight regions. In practice, "tight" means many keys share a long common prefix — they differ only in the last few bits. Examples include URLs with shared domain prefixes, file paths with shared directory prefixes, consecutive timestamps, and geospatial S2 cell IDs.

When a cluster is isolated into its own sub-filter, the adaptive filter sees only the local spread $M = \lceil \log_2(\max - \min) \rceil$ of that cluster (Section 4.3). Since $M_{\text{cluster}} \ll K$, exact mode triggers:

no hash, FPR = 0, and the universe size shrinks to fit just the cluster's range. The tighter the cluster, the fewer bits the ERE needs.

This is the central advantage over a single global filter: instead of one universe spanning the entire key space (where $M > K$ forces hashing and nonzero FPR), we get many small exact universes — each with zero false positives (Figure 5.2).

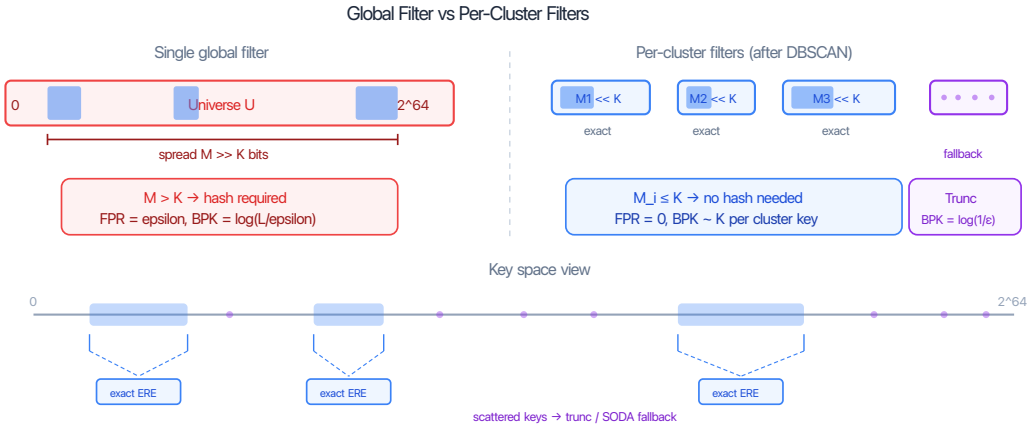

Figure 5.2: Global filter vs per-cluster exact filters. A single global universe forces hashing (nonzero FPR). Isolating each cluster into its own universe enables exact mode (FPR = 0) with fewer bits per key.

The Role of Segmentation

The quality of the hybrid filter depends directly on the quality of segmentation — specifically, on how well it separates keys that benefit from exact mode from those that should go to the fallback. Assigning all keys to a single cluster is trivial but useless: the spread is too large for exact mode. Conversely, creating too many small clusters wastes metadata overhead. The segmentation must find the right boundary: tight clusters where exact mode triggers (spread $< 2^K$), while routing sparse outliers to a fallback filter where hashing handles them efficiently.

The rest of this chapter explores two segmentation strategies of increasing sophistication: a simple gap-percentile heuristic (Section 5.1) and a density-based 1D DBSCAN approach (Section 5.2) that addresses its limitations.

5.1 Gap-Percentile Segmentation

The first hybrid approach splits keys into dense clusters and sparse remainder using gap-percentile detection: large gaps between sorted keys mark cluster boundaries.

Given n sorted keys, compute gaps $g_i = \text{key}_{i+1} - \text{key}_i$. The 95th-percentile gap (computed via quickselect) becomes the threshold τ. Then:

1. **Split** at every position where $g_i > \tau$.
2. **Filter:** segments with $\geq 1\%$ of n keys become clusters.
3. **Fallback:** remaining keys go to a truncation filter.

Each cluster is built as an adaptive filter (Section 4.3) — if the cluster's spread fits in K bits, exact mode triggers (FPR = 0). Otherwise SODA mode.

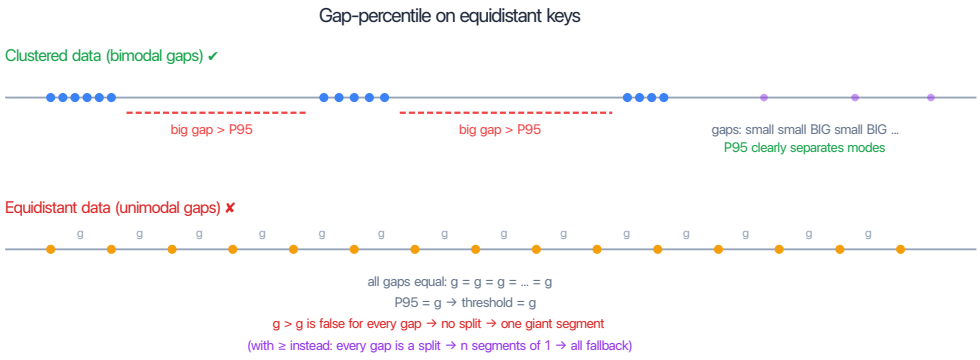

Figure 5.3: Gap-percentile failure on equidistant data: all gaps are equal, so P_{95} equals the common gap and no split points are found.

Limitations.

- **Equidistant data:** when all gaps are equal (sequential keys), the gap distribution has a single mode — there is no "large gap" to split at. The P95 threshold equals the common gap, and strict inequality fails for all gaps, producing one giant segment.
- **No merging:** two adjacent small segments that together would form a valid cluster are never joined.
- **Fallback is always truncation:** if fallback keys have small gaps relative to the phantom size, truncation produces high FPR (Section 4.2.3).

5.2 1D DBSCAN Segmentation (Scan-ARE)

The improved hybrid addresses all three limitations above using density-based clustering.

5.2.1 1D DBSCAN in $O(n)$

DBSCAN [Ester et al., 1996] is normally $O(n^2)$ due to neighbor search. On sorted 1D data, the δ-neighborhood of any point is an interval, so the algorithm reduces to two-pointer sweeps:

1. **Forward sweep:** for each key$[i]$, advance the left pointer while key$[i]$ − key[left] $> \delta$. If the window has $\geq$ `minPts` points, key$[i]$ is a core point.
2. **Reverse sweep:** same logic right-to-left (the forward pass only marks the rightmost points of each dense window).
3. **Core runs:** contiguous core points with gap $\leq \delta$ form cluster cores.
4. **Merge:** adjacent cores within δ of each other are joined.
5. **Border expansion:** non-core points within δ of a core point (*border points*) join the nearest cluster.
6. **Post-filter:** clusters with fewer than `minClusterSize` keys dissolve back to fallback.

Parameter	Value	Role
`minPts`	10	DBSCAN core threshold
`minClusterSize`	256	Post-filter: clusters below this dissolve

Table 5.1: DBSCAN parameters. Two separate parameters — unlike classic DBSCAN which conflates core density and cluster size.

5.2.2 Choosing δ From Problem Parameters

$$\delta = c \cdot \frac{\mathcal{L}}{\varepsilon}, \quad c = 10 \tag{5.1}$$

From the exact mode analysis (Equation 4.2): exact mode triggers when the average gap $< \mathcal{L}/\varepsilon$. So $\delta = 10 \cdot \mathcal{L}/\varepsilon$ means: regions $10\times$ denser than the exact-mode threshold get clustered.

Distribution	Typical gap	Gap vs δ	Result
Cluster (internal gap $\sim$100)	100	$\ll \delta$	Cluster found
Uniform 64-bit (avg gap $\sim 2^{44}$)	$\sim 10^{13}$	$\gg \delta$	All fallback
Sequential (gap = 1000)	1000	$\ll \delta$	One big cluster

Table 5.2: Sanity check for $\mathcal{L} = 128$, $\varepsilon = 10^{-3}$, $\delta = 1.28 \times 10^6$. No data statistics or percentiles needed — δ is derived purely from $\mathcal{L}$ and ε.

5.2.3 Dual Fallback

Keys not assigned to any cluster go to a fallback filter. The choice between truncation and SODA is made automatically:

1. Let spread = max(fallback keys)−min(fallback keys). Compute phantom size: phantom_size = spread/2^K.
2. Compute the 5th-percentile gap (P_5) of fallback keys via quickselect.
3. If $P_5 >$ phantom_size: **truncation** (safe, saves $\log_2(\mathcal{L})$ BPK).
4. If $P_5 \leq$ phantom_size: **adaptive/SODA** (guaranteed FPR).

5.2.4 Query

Clusters are sorted by $[\min, \max]$. For a query $[a, b]$:

1. Binary search over clusters intersecting $[a, b]$ — $O(\log C)$ where C is the number of clusters.
2. Check each intersecting cluster (typically 0–1).
3. Check the fallback filter.
4. Return "empty" only if all sub-filters agree.

5.2.5 FPR Guarantee

Each sub-filter independently targets FPR $\leq \varepsilon$. By a union bound, if a query touches at most C sub-filters, the overall FPR $\leq C\varepsilon$. In practice most queries touch only one sub-filter (either one cluster or just the fallback), so $C = 1$ and FPR $\approx \varepsilon$.

Chapter 6

Experimental Evaluation

6.1 Benchmark Setup

All experiments use 60-bit keys from the SOSD benchmark suite [Marcus et al., 2020]: Facebook user IDs, Wikipedia edit timestamps, OpenStreetMap cell IDs, and Books ISBNs. Synthetic distributions (uniform, clustered) supplement the real-world datasets.

Competing implementations:

- **Grafite** [Boffa et al., 2024]: information-theoretically optimal range filter (SIGMOD 2024).
- **SNARF** [Vaidya et al., 2022]: learned CDF-based range filter.
- **SuRF** [Zhang et al., 2018]: succinct range filter with three variants (base, hash suffix, real suffix).
- **Bloom ARE**: trivial baseline — standard Bloom filter with $\mathcal{L}$ point lookups per range query.

All filters are evaluated on FPR-vs-BPK tradeoff curves, measured over 10^6 random empty-range queries per configuration.

6.2 Large-Range Performance: $\mathcal{L} = 65536$

The combination of truncation-based hashing and exact-mode clusters enables handling very large range queries that would be impractical for other approaches.

At $\mathcal{L} = 65536$, non-hybrid filters face a fundamental problem: the fingerprint length $K = \lceil \log_2(n \cdot \mathcal{L}/\varepsilon) \rceil$ must grow by $\log_2(\mathcal{L}) = 16$ bits to maintain the same FPR. For SODA, this means 16 extra BPK. For Bloom, $\mathcal{L} = 65536$ point lookups per query is impractical.

Scan-ARE sidesteps this on two levels:

Exact clusters. Each cluster stores its $[\min, \max]$ bounds. If a query range $[a, b]$ overlaps a cluster's bounds, at least one stored key is inside $[a, b]$ — that is a true positive, not a false positive. If $[a, b]$ does not overlap any cluster, the cluster contributes nothing. If $[a, b]$ falls entirely within a cluster, the sub-filter is exact (Section 4.3), so false positives are impossible. Either way, exact clusters produce zero false positives regardless of $\mathcal{L}$.

Truncation fallback. The sparse fallback uses truncation hashing (Section 4.2), where phantom points form a block adjacent to each stored key (Figure 4.4). A false positive can only occur when a query *endpoint* a or b lands inside a phantom block — if a stored key lies inside $[a, b]$, that is a true positive, not a false positive. Because phantoms do not scatter across the universe the way SODA phantoms do, the FPR of truncation is independent of the query range length $\mathcal{L}$. The fallback thus avoids the $\log_2(\mathcal{L})$ BPK penalty that SODA-based filters must pay.

6.2.1 SOSD Results ($n = 2^{24}$, $\mathcal{L} = 65536$)

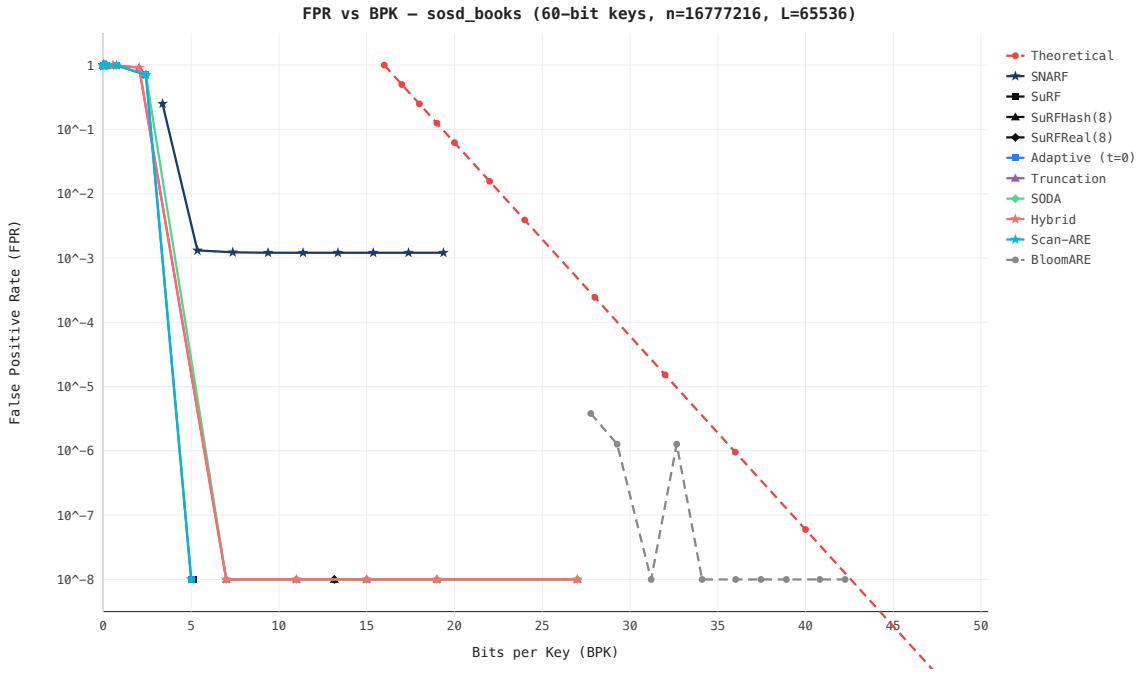

Figure 6.1: FPR vs BPK — SOSD Books (ISBN sequences, highly clustered), $n = 2^{24}$, $\mathcal{L} = 65536$. Scan-ARE achieves FPR $< 10^{-8}$ at $\sim$5 BPK; all non-hybrid filters degrade to FPR ≈ 1.

On both datasets, Scan-ARE achieves FPR $< 10^{-8}$ at $\sim$5 BPK, while Truncation, SODA, and Bloom all degrade to FPR ≈ 1 at comparable budgets. SNARF plateaus at FPR $\sim 10^{-3}$. Only Grafite remains competitive on Wiki ($\sim$10 BPK for FPR 10^{-2}), but at 2–3$\times$ the space of Scan-ARE.

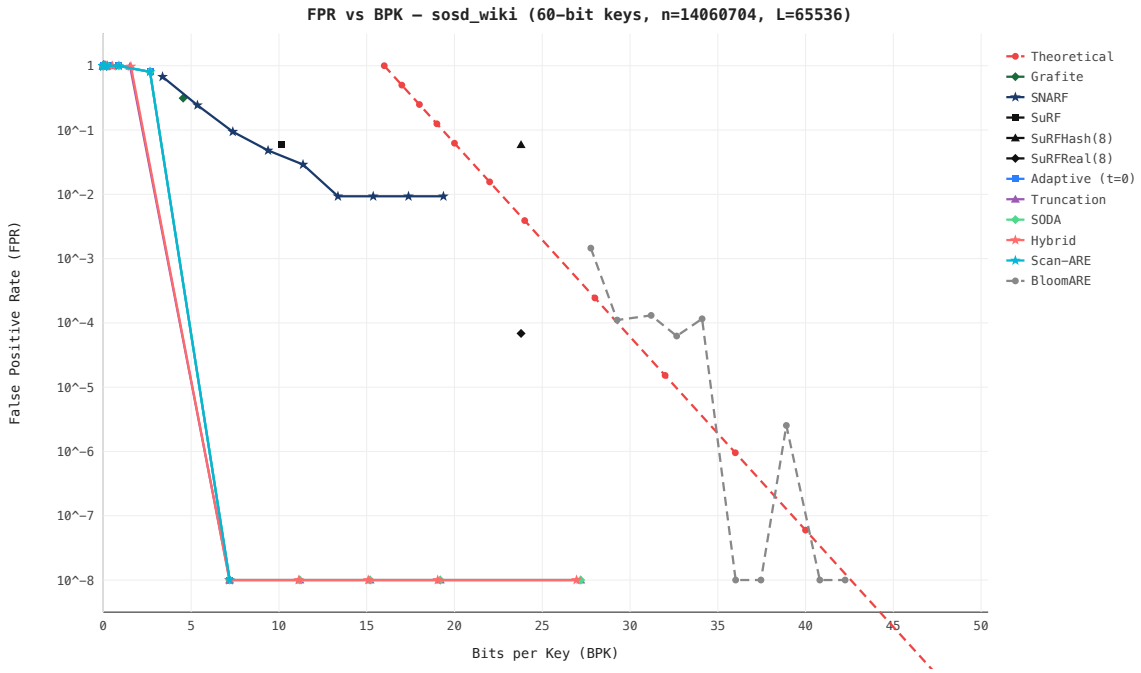

Figure 6.2: FPR vs BPK — SOSD Wiki (Wikipedia timestamps, moderately clustered), $n = 2^{24}$, $\mathcal{L} = 65536$. Scan-ARE dominates; only Grafite remains competitive at ∼10 BPK but with 2–3× more space.

6.3 BPK at Industry-Standard FPR Targets

In practice, storage systems target a specific FPR — typically 10^{-2} or 10^{-3} — and want to minimize the filter's memory footprint. The relevant question is: how many bits per key does each filter need to reach the target FPR?

We report this metric separately for each of the 9 benchmark distributions: 4 real-world (SOSD) and 5 synthetic. Each distribution exercises different filter strengths — a filter that excels on clustered data may be mediocre on uniform, and vice versa.

SOSD Distributions

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.1: BPK to reach target FPR — SOSD Facebook ($n = 2^{24}$, highly clustered).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.2: BPK to reach target FPR — SOSD Books ($n = 2^{24}$, clustered by publisher).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.3: BPK to reach target FPR — SOSD Wiki ($n = 2^{24}$, moderately clustered).

Synthetic Distributions

6.4 Summary of Results

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.4: BPK to reach target FPR — SOSD OSM ($n = 2^{24}$, geospatially clustered).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.5: BPK to reach target FPR — Clustered synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.6: BPK to reach target FPR — Uniform synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.7: BPK to reach target FPR — Spread synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.8: BPK to reach target FPR — Zipfian synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.9: BPK to reach target FPR — Temporal synthetic ($n = 2^{24}$).

Filter	BPK ($\mathcal{L}$=65536)	FPR	Distribution dependence
Scan-ARE	∼5	$< 10^{-8}$	Best on clustered data
Grafite	∼10	$\sim 10^{-2}$	Distribution-independent
SNARF	∼28	$\sim 10^{-3}$	Learned, data-dependent
SODA	∼5	≈ 1 (degenerate)	Distribution-independent
Truncation	∼3	≈ 1 (degenerate)	Fails on clustered data
Bloom	∼30	$\sim 10^{-4}$	Distribution-independent

Table 6.10: Comparison at $n = 2^{24}$, $\mathcal{L} = 65536$ on SOSD Books. Scan-ARE dominates the Pareto frontier by exploiting cluster density for exact-mode sub-filters.

Chapter 7

Conclusion

Bibliography

Antonio Boffa, Davide Cenzato, Paolo Ferragina, and Giorgio Vinciguerra. Grafite: Taming adversarial queries with optimal range filters. In *Proceedings of the 2024 International Conference on Management of Data (SIGMOD '24)*. ACM, 2024.

Peter Elias. Efficient storage and retrieval by content and address of static files. *Journal of the ACM*, 21(2):246–260, 1974.

Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD '96)*, pages 226–231, 1996.

Robert M Fano. On the number of bits sufficient to implement an associative memory. In *Memorandum 61, Project MAC, MIT*, 1971.

Paolo Ferragina and Giorgio Vinciguerra. The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds. *Proceedings of the VLDB Endowment*, 13(8):1162–1175, 2020.

Google. S2 Geometry Library, 2017. URL `https://s2geometry.io`. Accessed: 2026-03-25.

Mayank Goswami, Allan Grønlund Jørgensen, Kasper Green Larsen, and Rasmus Pagh. Approximate range emptiness in constant time and optimal space. In *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '15)*, pages 848–858. SIAM, 2015.

Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Tim Kraska, and Alfons Kemper. SOSD: A benchmark for learned indexes. *NeurIPS Workshop on Machine Learning for Systems*, 2020. URL `https://github.com/learnedsystems/SOSD`.

Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomization and Probabilistic Techniques in Algorithms and Data Structures*. Cambridge University Press, 2 edition, 2017.

Kapil Vaidya, Subhadeep Chatterjee, Eric Knorr, Michael Mitzenmacher, Stratos Idreos, and Tim Kraska. SNARF: A learning-enhanced range filter. *Proceedings of the VLDB Endowment*, 15(8):1632–1644, 2022.

Huanchen Zhang, Hyeontaek Lim, Viktor Leis, David G Andersen, Michael Kaminsky, Kimberly Keeton, and Andrew Pavlo. SuRF: Practical range query filtering with fast succinct tries. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*, pages 323–336. ACM, 2018.